Least mean squares filter

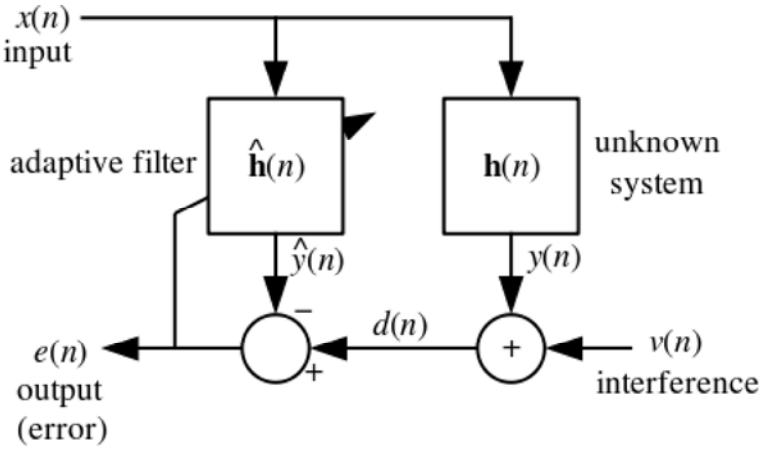
From Wikipedia, the free encyclopedia

Least mean squares (LMS) algorithms are a class of adaptive filter used to mimic a desired filter by finding the filter coefficients that relate to producing the least mean squares of the error signal (difference between the desired and the actual signal). It is a stochastic gradient descent method in that the filter is only adapted based on the error at the current time. It was invented in 1960 by Stanford University professor Bernard Widrow and his first Ph.D. student, Ted Hoff.

Contents

- 1 Problem formulation
  - 1.1 Relationship to the least squares filter
  - 1.2 Definition of symbols
- 2 Idea
- 3 Derivation
- 4 Simplifications
- 5 LMS algorithm summary
- 6 Convergence and stability in the mean
- 7 Normalised least mean squares filter (NLMS)
  - 7.1 Optimal learning rate
  - 7.2 Proof
- 8 See also
- 9 References
- 10 External links

Problem formulation



Relationship to the least squares filter

The realization of the causal Wiener filter looks a lot like the solution to the least squares estimate, except in the signal processing domain. The least squares solution, for input matrix $\mathbf{x}$ and output vector $\mathbf{y}$ is

$$\hat{\boldsymbol{\beta}} = (\mathbf{X}^\mathbf{T}\mathbf{X})^{-1}\mathbf{X}^\mathbf{T}\boldsymbol{y}.$$

The FIR least mean squares filter is related to the Wiener filter, but minimizing the error criterion of the former does not rely on cross-correlations or auto-correlations. Its solution converges to the Wiener filter solution. Most linear adaptive filtering problems can be formulated using the block diagram above. That is, an unknown system $\mathbf{h}(n)$ is to

be identified and the adaptive filter attempts to adapt the filter $\hat{\mathbf{h}}(n)$ to make it as close as possible to $\mathbf{h}(n)$, while using only observable signals $x(n)$, $d(n)$ and $e(n)$; but $y(n)$, $v(n)$ and $h(n)$ are not directly observable. Its solution is closely related to the Wiener filter.

Definition of symbols

n is the number of the current input sample
p is the number of filter taps
$\{\cdot\}^H$ (Hermitian transpose or conjugate transpose)
$\mathbf{x}(n) = [x(n), x(n-1), \ldots, x(n-p+1)]^T$
$\mathbf{h}(n) = [h_0(n), h_1(n), \ldots, h_{p-1}(n)]^T, \quad \mathbf{h}(n) \in \mathbb{C}^p$
$y(n) = \mathbf{h}^H(n) \cdot \mathbf{x}(n)$
$d(n) = y(n) + \nu(n)$
$\hat{\mathbf{h}}(n)$ estimated filter; interpret as the estimation of the filter coefficients after n samples
$e(n) = d(n) - \hat{y}(n) = d(n) - \hat{\mathbf{h}}^H(n) \cdot \mathbf{x}(n)$

Idea

The basic idea behind LMS filter is to approach the optimum filter weights $(R^{-1}P)$, by updating the filter weights in a manner to converge to the optimum filter weight. The algorithm starts by assuming small weights (zero in most cases) and, at each step, by finding the gradient of the mean square error, the weights are updated. That is, if the MSE-gradient is positive, it implies, the error would keep increasing positively, if the same weight is used for further iterations, which means we need to reduce the weights. In the same way, if the gradient is negative, we need to increase the weights. So, the basic weight update equation is :

$W_{n+1} = W_n - \mu \nabla \varepsilon[n]$,

where ε represents the mean-square error. The negative sign indicates that, we need to change the weights in a direction opposite to that of the gradient slope.

The mean-square error, as a function of filter weights is a quadratic function which means it has only one extremum, that minimises the mean-square error, which is the optimal weight. The LMS thus, approaches towards this optimal weights by ascending/descending down the mean-square-error vs filter weight curve.

Derivation

The idea behind LMS filters is to use steepest descent to find filter weights $\mathbf{h}(n)$ which minimize a cost function. We start by defining the cost function as

$$C(n) = E\left\{|e(n)|^2\right\}$$

where $e(n)$ is the error at the current sample n and $E\{\cdot\}$ denotes the expected value.

This cost function ($C(n)$) is the mean square error, and it is minimized by the LMS. This is where the LMS gets its name. Applying steepest descent means to take the partial derivatives with respect to the individual entries of the filter coefficient (weight) vector

$$\nabla_{\hat{\mathbf{h}}^H} C(n) = \nabla_{\hat{\mathbf{h}}^H} E\left\{e(n)\, e^*(n)\right\} = 2E\left\{\nabla_{\hat{\mathbf{h}}^H}(e(n))\, e^*(n)\right\}$$

where ∇ is the gradient operator

$$\nabla_{\hat{\mathbf{h}}^H}(e(n)) = \nabla_{\hat{\mathbf{h}}^H}\left(d(n) - \hat{\mathbf{h}}^H \cdot \mathbf{x}(n)\right) = -\mathbf{x}(n)$$
$$\nabla C(n) = -2E\left\{\mathbf{x}(n)\, e^*(n)\right\}$$

Now, $\nabla C(n)$ is a vector which points towards the steepest ascent of the cost function. To find the minimum of the

cost function we need to take a step in the opposite direction of $\nabla C(n)$. To express that in mathematical terms

$$\hat{\mathbf{h}}(n+1) = \hat{\mathbf{h}}(n) - \frac{\mu}{2}\nabla C(n) = \hat{\mathbf{h}}(n) + \mu\, E\left\{\mathbf{x}(n)\, e^*(n)\right\}$$

where $\frac{\mu}{2}$ is the step size(adaptation constant). That means we have found a sequential update algorithm which minimizes the cost function. Unfortunately, this algorithm is not realizable until we know $E\left\{\mathbf{x}(n)\, e^*(n)\right\}$.

Generally, the expectation above is not computed. Instead, to run the LMS in an online (updating after each new sample is received) environment, we use an instantaneous estimate of that expectation. See below.

Simplifications

For most systems the expectation function $E\left\{\mathbf{x}(n)\, e^*(n)\right\}$ must be approximated. This can be done with the following unbiased estimator

$$\hat{E}\left\{\mathbf{x}(n)\, e^*(n)\right\} = \frac{1}{N}\sum_{i=0}^{N-1}\mathbf{x}(n-i)\, e^*(n-i)$$

where N indicates the number of samples we use for that estimate. The simplest case is $N = 1$

$$\hat{E}\left\{\mathbf{x}(n)\, e^*(n)\right\} = \mathbf{x}(n)\, e^*(n)$$

For that simple case the update algorithm follows as

$$\hat{\mathbf{h}}(n+1) = \hat{\mathbf{h}}(n) + \mu\mathbf{x}(n)\, e^*(n)$$

Indeed this constitutes the update algorithm for the LMS filter.

LMS algorithm summary

The LMS algorithm for a pth order algorithm can be summarized as

Parameters:	$p =$ filter order
	$\mu =$ step size
Initialisation:	$\hat{\mathbf{h}}(0) = \mathbf{zeros}(p)$
Computation:	For $n = 0, 1, 2, \ldots$

$$\mathbf{x}(n) = \left[x(n), x(n-1), \ldots, x(n-p+1)\right]^T$$

$$e(n) = d(n) - \hat{\mathbf{h}}^H(n)\mathbf{x}(n)$$
$$\hat{\mathbf{h}}(n+1) = \hat{\mathbf{h}}(n) + \mu\, e^*(n)\mathbf{x}(n)$$

Convergence and stability in the mean

As the LMS algorithm does not use the exact values of the expectations, the weights would never reach the optimal weights in the absolute sense, but a convergence is possible in mean. That is, even though the weights may change by small amounts, it changes about the optimal weights. However, if the variance with which the weights change, is large, convergence in mean would be misleading. This problem may occur, if the value of step-size μ is not chosen properly.

If μ is chosen to be large, the amount with which the weights change depends heavily on the gradient estimate, and so the weights may change by a large value so that gradient which was negative at the first instant may now become positive. And at the second instant, the weight may change in the opposite direction by a large amount because of the negative gradient and would thus keep oscillating with a large variance about the optimal weights. On the other hand if μ is chosen to be too small, time to converge to the optimal weights will be too large.

Thus, an upper bound on μ is needed which is given as $0 < \mu < \frac{2}{\lambda_{\max}}$

where $\lambda_{\max}$ is the greatest eigenvalue of the autocorrelation matrix $\mathbf{R} = E\{\mathbf{x}(n)\mathbf{x}^H(n)\}$. If this condition is not fulfilled, the algorithm becomes unstable and $\hat{h}(n)$ diverges.

Maximum convergence speed is achieved when

$$\mu = \frac{2}{\lambda_{\max} + \lambda_{\min}},$$

where $\lambda_{\min}$ is the smallest eigenvalue of R. Given that μ is less than or equal to this optimum, the convergence speed is determined by $\lambda_{\min}$, with a larger value yielding faster convergence. This means that faster convergence can be achieved when $\lambda_{\max}$ is close to $\lambda_{\min}$, that is, the maximum achievable convergence speed depends on the eigenvalue spread of $\mathbf{R}$.

A white noise signal has autocorrelation matrix $\mathbf{R} = \sigma^2\mathbf{I}$ where σ^2 is the variance of the signal. In this case all eigenvalues are equal, and the eigenvalue spread is the minimum over all possible matrices. The common interpretation of this result is therefore that the LMS converges quickly for white input signals, and slowly for colored input signals, such as processes with low-pass or high-pass characteristics.

It is important to note that the above upperbound on μ only enforces stability in the mean, but the coefficients of $\hat{h}(n)$ can still grow infinitely large, i.e. divergence of the coefficients is still possible. A more practical bound is

$$0 < \mu < \frac{2}{\operatorname{tr}[\mathbf{R}]},$$

where $\operatorname{tr}[\mathbf{R}]$ denotes the trace of $\mathbf{R}$. This bound guarantees that the coefficients of $\hat{h}(n)$ do not diverge (in practice, the value of μ should not be chosen close to this upper bound, since it is somewhat optimistic due to approximations and assumptions made in the derivation of the bound).

Normalised least mean squares filter (NLMS)

The main drawback of the "pure" LMS algorithm is that it is sensitive to the scaling of its input $x(n)$. This makes it very hard (if not impossible) to choose a learning rate μ that guarantees stability of the algorithm (Haykin 2002). The *Normalised least mean squares filter* (NLMS) is a variant of the LMS algorithm that solves this problem by normalising with the power of the input. The NLMS algorithm can be summarised as:

Parameters:	$p =$ filter order
	$\mu =$ step size
Initialization:	$\hat{\mathbf{h}}(0) = \mathbf{zeros}(p)$
Computation:	For $n = 0, 1, 2, \ldots$

$$\mathbf{x}(n) = [x(n), x(n-1), \ldots, x(n-p+1)]^T$$

$$e(n) = d(n) - \hat{\mathbf{h}}^H(n)\mathbf{x}(n)$$

$$\hat{\mathbf{h}}(n+1) = \hat{\mathbf{h}}(n) + \frac{\mu\, e^*(n)\mathbf{x}(n)}{\mathbf{x}^H(n)\mathbf{x}(n)}$$

Optimal learning rate

It can be shown that if there is no interference ($v(n) = 0$), then the optimal learning rate for the NLMS algorithm is

$$\mu_{opt} = 1$$

and is independent of the input $x(n)$ and the real (unknown) impulse response $\mathbf{h}(n)$. In the general case with

interference ($v(n) \neq 0$), the optimal learning rate is

$$\mu_{opt} = \frac{E\left[|y(n) - \hat{y}(n)|^2\right]}{E\left[|e(n)|^2\right]}$$

The results above assume that the signals $v(n)$ and $x(n)$ are uncorrelated to each other, which is generally the case in practice.

Proof

Let the filter misalignment be defined as $\Lambda(n) = \left|\mathbf{h}(n) - \hat{\mathbf{h}}(n)\right|^2$, we can derive the expected misalignment for the next sample as:

$$E\left[\Lambda(n+1)\right] = E\left[\left|\hat{\mathbf{h}}(n) + \frac{\mu\, e^*(n)\mathbf{x}(n)}{\mathbf{x}^H(n)\mathbf{x}(n)} - \mathbf{h}(n)\right|^2\right]$$
$$E\left[\Lambda(n+1)\right] = E\left[\left|\hat{\mathbf{h}}(n) + \frac{\mu\,\left(v^*(n) + y^*(n) - \hat{y}^*(n)\right)\mathbf{x}(n)}{\mathbf{x}^H(n)\mathbf{x}(n)} - \mathbf{h}(n)\right|^2\right]$$

Let $\delta = \hat{\mathbf{h}}(n) - \mathbf{h}(n)$ and $r(n) = \hat{y}(n) - y(n)$

$$E\left[\Lambda(n+1)\right] = E\left[\left|\delta(n) - \frac{\mu\,\left(v(n) + r(n)\right)\mathbf{x}(n)}{\mathbf{x}^H(n)\mathbf{x}(n)}\right|^2\right]$$
$$E\left[\Lambda(n+1)\right] = E\left[\left(\delta(n) - \frac{\mu\,\left(v(n) + r(n)\right)\mathbf{x}(n)}{\mathbf{x}^H(n)\mathbf{x}(n)}\right)^H\left(\delta(n) - \frac{\mu\,\left(v(n) + r(n)\right)\mathbf{x}(n)}{\mathbf{x}^H(n)\mathbf{x}(n)}\right)\right]$$

Assuming independence, we have:

$$E\left[\Lambda(n+1)\right] = \Lambda(n) + E\left[\left(\frac{\mu\,\left(v(n) - r(n)\right)\mathbf{x}(n)}{\mathbf{x}^H(n)\mathbf{x}(n)}\right)^H\left(\frac{\mu\,\left(v(n) - r(n)\right)\mathbf{x}(n)}{\mathbf{x}^H(n)\mathbf{x}(n)}\right)\right] - 2E\left[\frac{\mu|r(n)|^2}{\mathbf{x}^H(n)\mathbf{x}(n)}\right]$$
$$E\left[\Lambda(n+1)\right] = \Lambda(n) + \frac{\mu^2 E\left[|e(n)|^2\right]}{\mathbf{x}^H(n)\mathbf{x}(n)} - \frac{2\mu E\left[|r(n)|^2\right]}{\mathbf{x}^H(n)\mathbf{x}(n)}$$

The optimal learning rate is found at $\frac{dE\left[\Lambda(n+1)\right]}{d\mu} = 0$, which leads to:

$$2\mu E\left[|e(n)|^2\right] - 2E\left[|r(n)|^2\right] = 0$$
$$\mu = \frac{E\left[|r(n)|^2\right]}{E\left[|e(n)|^2\right]}$$

See also

- Recursive least squares
- For statistical techniques relevant to LMS filter see Least squares.
- Similarities between Wiener and LMS
- Multidelay block frequency domain adaptive filter
- Zero forcing equalizer
- Kernel adaptive filter

References

- Monson H. Hayes: *Statistical Digital Signal Processing and Modeling,* Wiley, 1996, ISBN 0-471-59431-8
- Simon Haykin: *Adaptive Filter Theory,* Prentice Hall, 2002, ISBN 0-13-048434-2
- Simon S. Haykin, Bernard Widrow (Editor): *Least-Mean-Square Adaptive Filters,* Wiley, 2003, ISBN 0-471-21570-8
- Bernard Widrow, Samuel D. Stearns: *Adaptive Signal Processing,* Prentice Hall, 1985, ISBN 0-13-004029-0
- Weifeng Liu, Jose Principe and Simon Haykin: *Kernel Adaptive Filtering: A Comprehensive Introduction,* John Wiley, 2010, ISBN 0-470-44753-2
- Paulo S.R. Diniz: *Adaptive Filtering: Algorithms and Practical Implementation,* Kluwer Academic Publishers, 1997, ISBN 0-7923-9912-9

External links

- LMS Algorithm in Adaptive Antenna Arrays (http://www.antenna-theory.com/arrays/weights/lms.php) www.antenna-theory.com
- LMS Noise cancellation demo (http://www.advsolned.com/example_ale_nc.html) www.advsolned.com

Retrieved from "https://en.wikipedia.org/w/index.php?title=Least_mean_squares_filter&oldid=730206508"

Categories: Digital signal processing | Filter theory | Stochastic algorithms

---

- This page was last modified on 17 July 2016, at 13:23.
- Text is available under the Creative Commons Attribution-ShareAlike License; additional terms may apply. By using this site, you agree to the Terms of Use and Privacy Policy. Wikipedia® is a registered trademark of the Wikimedia Foundation, Inc., a non-profit organization.